

✓ Welcome to Colab!

Google Colab is available in VS Code!



Try the new [Google Colab extension](#) for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the **Extensions** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the **Select kernel** button in the top right.
- Click **Colab** and then select your desired runtime, sign in with your Google Account and you're all set!

See more details in our [announcement blog here](#).

🎁 Free-of-charge Pro plan for Gemini and Colab for US university students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free-of-charge offer at gemini.google/students.

Get the Colab free-of-charge offer at colab.research.google.com/signup.

Terms apply.

✓ Access popular AI models via Google Colab-AI without an API key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to [getting started with Google Colab AI](#).

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

✓ Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code and audio.

How to get started

- Go to [Google AI Studio](#) and log in with your Google Account.
- [Create an API key](#).
- Use a quickstart for [Python](#) or call the REST API using [curl](#).

Discover Gemini's advanced capabilities

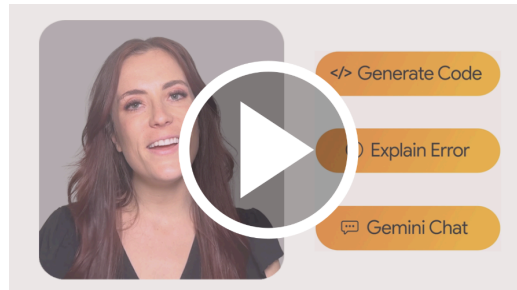
- Play with Gemini [multimodal outputs](#), mixing text and images in an iterative way.
- Discover the [multimodal Live API](#) (demo [here](#)).
- Learn how to [analyse images and detect items in your pictures](#) using Gemini (bonus, there's a [3D version](#) as well!).
- Unlock the power of the [Gemini thinking model](#), capable of solving complex tasks with its inner thoughts.

Explore complex use cases

- Use [Gemini grounding capabilities](#) to create a report on a company based on what the model can find on the Internet.
- Extract [invoices and form data from PDFs](#) in a structured way.
- Create [illustrations based on a whole book](#) using Gemini large context window and Imagen.

To learn more, take a look at the [Gemini cookbook](#) or visit the [Gemini API documentation](#).

Colab now has AI features powered by [Gemini](#). The video below provides information on how to use these features, whether you're new to Python or a seasoned veteran.



What is Colab?

Colab, or 'Colaboratory', allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab features you may have missed](#) to learn more or just get started below!

✓ Getting started

The document that you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

86400

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut 'Command/Ctrl+Enter'. To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

604800

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To find out more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [Create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To find out more about the Jupyter project, see jupyter.org.

✓ Data science

With Colab you can harness the full power of popular Python libraries to analyse and visualise data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualise it. To edit the code, just click the cell and start editing.

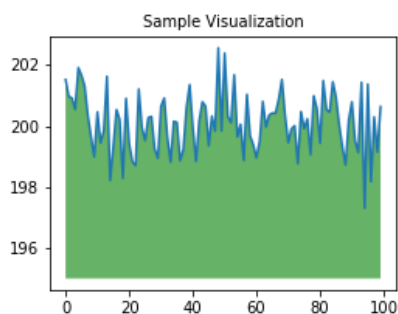
You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from GitHub and many other sources. To find out more about importing data, and how Colab can be used for data science, see the links below under [Working with data](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"!!![{alt}]({image})"))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning that you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10-minute guide](#) or [US stock market data analysis demo](#).

✓ Machine learning

With Colab you can import an image dataset, train an image classifier on it and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

✓ More resources

Working with notebooks in Colab

- [Overview of Colab](#)
- [Guide to markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with data

- [Loading data: Drive, Sheets and Google Cloud Storage](#)
- [Charts: visualising data](#)
- [Getting started with BigQuery](#)

Machine learning

These are a few of the notebooks related to machine learning, including Google's online machine learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Getting started with cuML's accelerator mode](#)

Using accelerated hardware

- [Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API](#)
- [Train a Vision Transformer \(ViT\) for image classification with JAX](#)
- [Text classification with a transformer language model using JAX](#)

✓ Featured examples

- [Train a miniGPT language model with JAX AI stack](#)
- [LoRA/QLoRA finetuning for LLM using Tunix](#)
- [Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA](#)
- [Loading Hugging Face transformers checkpoints](#)
- [8-bit integer quantisation in Keras](#)
- [Float8 training and inference with a simple transformer model](#)
- [Pretraining a transformer from scratch with KerasHub](#)
- [Simple MNIST convnet](#)
- [Image classification from scratch using Keras 3](#)
- [Image classification with KerasHub](#)

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('/content/Apps_data+(1) (1).csv')
df
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25M	50,000,000+	Free	0	Teen
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8M	100,000+	Free	0	Everyone
...	...	...	...	...	...	...	...	...	...
10836	Sya9a Maroc - FR	FAMILY	4.5	38	53M	5,000+	Free	0	Everyone
10837	Fr. Mike Schmitz Audio Teachings	FAMILY	5.0	4	3.6M	100+	Free	0	Everyone
10838	Parkinson Exercices FR	MEDICAL	NaN	3	9.5M	1,000+	Free	0	Everyone
10839	The SCP Foundation DB fr nn5n	BOOKS_AND_REFERENCE	4.5	114	Varies with device	1,000+	Free	0	Mature 17+
10840	iHoroscope - 2018 Daily Horoscope & Astrology	LIFESTYLE	4.5	398307	19M	10,000,000+	Free	0	Everyone

10841 rows × 13 columns

Next steps:

Generate code with df

New interactive sheet

```
df.sample(5)
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated
6496	Basket Manager 2017 Pro	SPORTS	4.7	163	4.6M	1,000+	Paid	\$0.99	Everyone	Sports	Oct 6, 2017
6495	Word Hunt	GAME	4.9	59	21M	500+	Free	0	Everyone	Word	June 2, 2017
9750	ER Blood Draw Doctor & Surgeon	FAMILY	4.0	154	58M	50,000+	Free	0	Teen	Educational	June 2, 2017
2045	Barbie™ Fashion Closet	FAMILY	4.1	68286	85M	10,000,000+	Free	0	Everyone	Casual;Creativity	July 2, 2017
319	Children's cartoons (Mithu-Mina-Raju)	COMICS	4.6	279	3.3M	100,000+	Free	0	Everyone	Comics	Dec 18, 2017

```
df.shape
```

```
(10841, 13)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   App                    10841 non-null  object
1   Category               10841 non-null  object
2   Rating                 9367 non-null   float64
3   Reviews                10841 non-null  object
4   Size                   10841 non-null  object
5   Installs               10841 non-null  object
6   Type                   10840 non-null  object
7   Price                  10841 non-null  object
8   Content Rating         10840 non-null  object
9   Genres                 10841 non-null  object
10  Last Updated           10841 non-null  object
11  Current Ver            10833 non-null  object
12  Android Ver            10838 non-null  object
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
```

```
df.describe(include='all')
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated
count	10841	10841	9367.000000	10841	10841	10841	10840	10841	10840	10841	10841
unique	9660	34	NaN	6002	462	22	3	93	6	120	137
top	ROBLOX	FAMILY	NaN	0	Varies with device	1,000,000+	Free	0	Everyone	Tools	August 3, 2019
freq	9	1972	NaN	596	1695	1579	10039	10040	8714	842	32
mean	NaN	NaN	4.193338	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	0.537431	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	4.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	4.300000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	4.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	19.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
a=df.drop(columns='App')
a.columns
```

```
Index(['Category', 'Rating', 'Reviews', 'Size', 'Installs', 'Type', 'Price',
      'Content Rating', 'Genres', 'Last Updated', 'Current Ver',
      'Android Ver'],
      dtype='object')
```

```
df.duplicated().sum()
```

```
np.int64(483)
```

```
df.drop_duplicates(inplace=True)
df.duplicated().sum()
```

```
np.int64(0)
```

```
df['Category'].unique()
```

```
array(['ART_AND_DESIGN', 'AUTO_AND_VEHICLES', 'BEAUTY',
      'BOOKS_AND_REFERENCE', 'BUSINESS', 'COMICS', 'COMMUNICATION',
      'DATING', 'EDUCATION', 'ENTERTAINMENT', 'EVENTS', 'FINANCE',
      'FOOD_AND_DRINK', 'HEALTH_AND_FITNESS', 'HOUSE_AND_HOME',
      'LIBRARIES_AND_DEMO', 'LIFESTYLE', 'GAME', 'FAMILY', 'MEDICAL',
      'SOCIAL', 'SHOPPING', 'PHOTOGRAPHY', 'SPORTS', 'TRAVEL_AND_LOCAL',
      'TOOLS', 'PERSONALIZATION', 'PRODUCTIVITY', 'PARENTING', 'WEATHER',
      'VIDEO_PLAYERS', 'NEWS_AND_MAGAZINES', 'MAPS_AND_NAVIGATION',
      '1.9'], dtype=object)
```

```
df=df.drop(df[df['Category']=='1.9'].index)
```

```
df['Rating'].isnull().sum()
```

```
np.int64(1465)
```

```
df.dropna(subset='Rating',inplace=True)
```

```
df[['Rating']]
```

	Rating
0	4.1
1	3.9
2	4.7
3	4.5
4	4.3
...	...
10834	4.0
10836	4.5
10837	5.0
10839	4.5
10840	4.5

8892 rows × 1 columns

```
df['Rating'].isnull().sum()
```

```
np.int64(0)
```

```
df['Rating_category']=df['Rating'].apply (lambda a:'high' if a>3.5 else 'low')
```

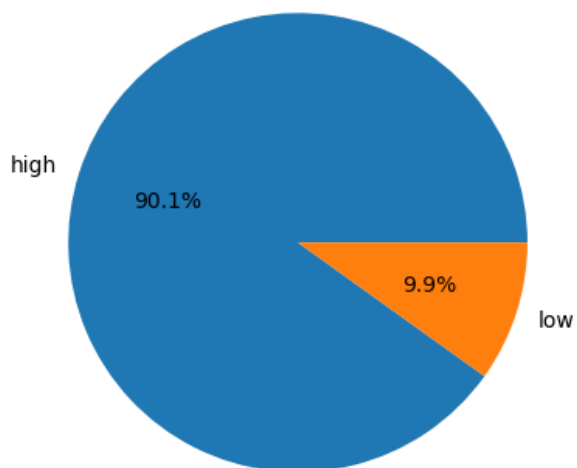
```
df['Rating_category']
```

	Rating_category
0	high
1	high
2	high
3	high
4	high
...	...
10834	high
10836	high
10837	high
10839	high
10840	high

8892 rows × 1 columns

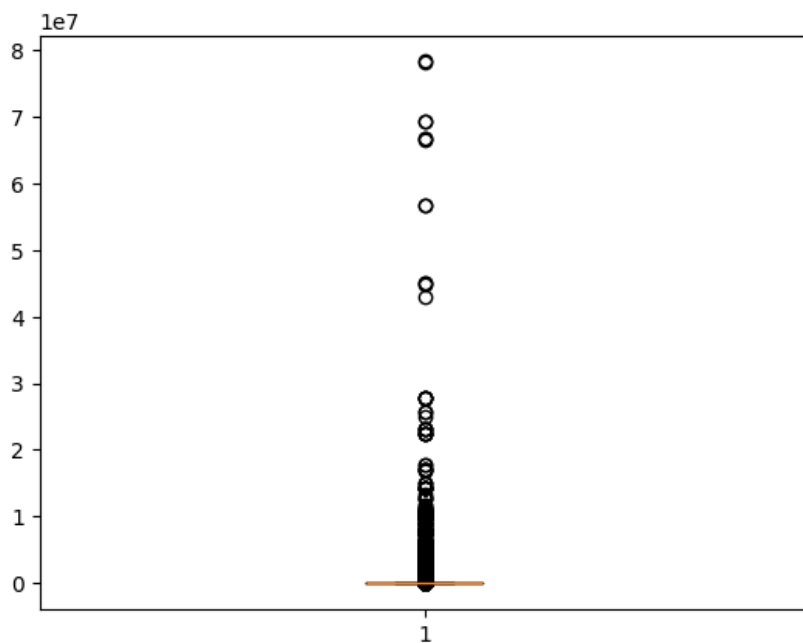
dtype: object

```
plt.pie(df.groupby(['Rating_category'])['Rating_category'].count(),autopct='%1.1f%%',labels=df.groupby(  
plt.show()
```

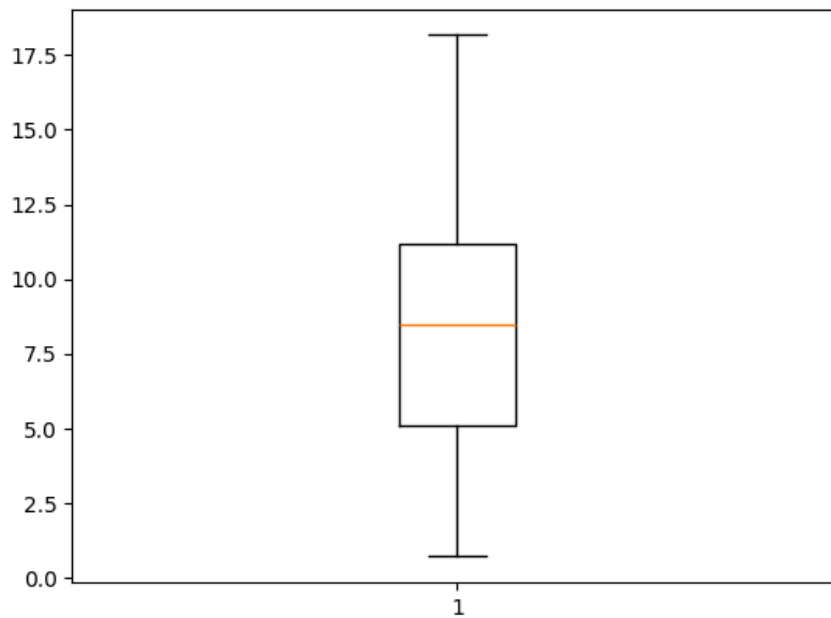



```
df['Reviews']=pd.to_numeric(df['Reviews'],errors='coerce')
```

```
plt.boxplot(df['Reviews'].dropna())  
plt.show()
```



```
df['Reviews']=np.log1p(df['Reviews'])  
plt.boxplot(df['Reviews'].dropna())  
plt.show()
```



```
df['Size']=df['Size'].str.replace('M','*10**6').str.replace('k','*1000')
df['Size']
```

	Size
0	19*10**6
1	14*10**6
2	8.7*10**6
3	25*10**6
4	2.8*10**6
...	...
10834	2.6*10**6
10836	53*10**6
10837	3.6*10**6
10839	Varies with device
10840	19*10**6

8892 rows × 1 columns

dtype: object

```
df=df.drop(df[df['Size']=='Varies with device'].index)
df
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	5.075174	19*10**6	10,000+	Free	0	Everyone
1	Coloring book moana	ART_AND_DESIGN	3.9	6.875232	14*10**6	500,000+	Free	0	Everyone
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	11.379520	8.7*10**6	5,000,000+	Free	0	Everyone
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	12.281389	25*10**6	50,000,000+	Free	0	Teen
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	6.875232	2.8*10**6	100,000+	Free	0	Everyone
...	...	...	...	...	...	...	...	...	...
10833	Chemin (fr)	BOOKS_AND_REFERENCE	4.8	3.806662	619*1000	1,000+	Free	0	Everyone
10834	FR Calculator	FAMILY	4.0	2.079442	2.6*10**6	500+	Free	0	Everyone
10836	Sya9a Maroc - FR	FAMILY	4.5	3.663562	53*10**6	5,000+	Free	0	Everyone
10837	Fr. Mike Schmitz Audio Teachings	FAMILY	5.0	1.609438	3.6*10**6	100+	Free	0	Everyone
10840	iHoroscope - 2018 Daily Horoscope & Astrology	LIFESTYLE	4.5	12.894981	19*10**6	10,000,000+	Free	0	Everyone

7424 rows × 14 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
print(df['Size'])
```

```
0      19000000.0
1      14000000.0
2       8700000.0
3      25000000.0
4      28000000.0
```

```
...
10833    619000.0
10834    2600000.0
10836    53000000.0
10837     3600000.0
10840    19000000.0
```

```
Name: Size, Length: 7424, dtype: float64
```

```
df['Installs'].unique()
```

```
array(['10,000+', '500,000+', '5,000,000+', '50,000,000+', '100,000+',
      '50,000+', '1,000,000+', '10,000,000+', '5,000+', '100,000,000+',
      '1,000+', '500,000,000+', '100+', '500+', '10+', '1,000,000,000+',
      '5+', '50+', '1+'], dtype=object)
```

```
df['Installs']=df['Installs'].str.replace(',','').astype(str)
```

```
df['Installs'].unique()
```

```
array(['10000+', '500000+', '5000000+', '50000000+', '100000+', '50000+',
      '1000000+', '10000000+', '5000+', '100000000+', '1000+',
      '500000000+', '100+', '500+', '10+', '1000000000+', '5+', '50+',
      '1+'], dtype=object)
```

```
df['Price'].unique()
```

```
array(['0', '$4.99', '$6.99', '$7.99', '$3.99', '$5.99', '$2.99', '$1.99',
      '$9.99', '$0.99', '$9.00', '$5.49', '$10.00', '$24.99', '$11.99',
      '$79.99', '$16.99', '$14.99', '$29.99', '$12.99', '$3.49',
      '$10.99', '$7.49', '$1.50', '$19.99', '$15.99', '$33.99', '$39.99',
      '$2.49', '$4.49', '$1.70', '$1.49', '$3.88', '$399.99', '$17.99',
      '$400.00', '$3.02', '$1.76', '$4.84', '$4.77', '$1.61', '$1.59',
      '$6.49', '$1.29', '$299.99', '$379.99', '$37.99', '$18.99',
      '$389.99', '$8.49', '$1.75', '$14.00', '$2.00', '$3.08', '$2.59',
      '$19.40', '$15.46', '$8.99', '$3.04', '$13.99', '$4.29', '$3.28',
      '$4.60', '$1.00', '$2.90', '$1.97', '$2.56', '$1.20'], dtype=object)
```

```
print(df['Price'])
```

```
0      0.0
1      0.0
2      0.0
3      0.0
4      0.0
...
10833  0.0
10834  0.0
10836  0.0
10837  0.0
10840  0.0
Name: Price, Length: 7424, dtype: float64
```

```
df=df.drop(columns=['Current Ver','App','Rating','Genres','Last Updated','Android Ver','Size'])
df
```

	Category	Reviews	Installs	Type	Price	Content Rating	Rating_category
0	ART_AND_DESIGN	5.075174	10000+	Free	0.0	Everyone	high
1	ART_AND_DESIGN	6.875232	500000+	Free	0.0	Everyone	high
2	ART_AND_DESIGN	11.379520	5000000+	Free	0.0	Everyone	high
3	ART_AND_DESIGN	12.281389	50000000+	Free	0.0	Teen	high
4	ART_AND_DESIGN	6.875232	100000+	Free	0.0	Everyone	high
...	...	...	...	...	...	...	...
10833	BOOKS_AND_REFERENCE	3.806662	1000+	Free	0.0	Everyone	high
10834	FAMILY	2.079442	500+	Free	0.0	Everyone	high
10836	FAMILY	3.663562	5000+	Free	0.0	Everyone	high
10837	FAMILY	1.609438	100+	Free	0.0	Everyone	high
10840	LIFESTYLE	12.894981	10000000+	Free	0.0	Everyone	high

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
num_cols=df.select_dtypes(include=np.number).columns
num_cols
```

```
Index(['Reviews', 'Price'], dtype='object')
```

```
catcols=df.select_dtypes(include='object').columns
catcols
```

```
Index(['Category', 'Installs', 'Type', 'Content Rating', 'Rating_category'], dtype='object')
```

```
catcols=df.select_dtypes(include='object').columns
catcols
```

```
Index(['Category', 'Installs', 'Type', 'Content Rating', 'Rating_category'], dtype='object')
```

df

	Category	Reviews	Installs	Type	Price	Content Rating	Rating_category
0	ART_AND_DESIGN	5.075174	10000+	Free	0.0	Everyone	high
1	ART_AND_DESIGN	6.875232	500000+	Free	0.0	Everyone	high
2	ART_AND_DESIGN	11.379520	5000000+	Free	0.0	Everyone	high
3	ART_AND_DESIGN	12.281389	50000000+	Free	0.0	Teen	high
4	ART_AND_DESIGN	6.875232	100000+	Free	0.0	Everyone	high
...	...	...	...	...	...	...	...
10833	BOOKS_AND_REFERENCE	3.806662	1000+	Free	0.0	Everyone	high
10834	FAMILY	2.079442	500+	Free	0.0	Everyone	high
10836	FAMILY	3.663562	5000+	Free	0.0	Everyone	high
10837	FAMILY	1.609438	100+	Free	0.0	Everyone	high
10840	LIFESTYLE	12.894981	10000000+	Free	0.0	Everyone	high

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
target=df[['Rating_category']]
target
```

	Rating_category	
0	high	
1	high	
2	high	
3	high	
4	high	
...	...	
10833	high	
10834	high	
10836	high	
10837	high	
10840	high	

7424 rows × 1 columns

Next steps:

[Generate code with target](#)[New interactive sheet](#)

```
independent_features=df.drop('Rating_category',axis=1)
independent_features
```

	Category	Reviews	Installs	Type	Price	Content Rating	
0	ART_AND_DESIGN	5.075174	10000+	Free	0.0	Everyone	
1	ART_AND_DESIGN	6.875232	500000+	Free	0.0	Everyone	
2	ART_AND_DESIGN	11.379520	5000000+	Free	0.0	Everyone	
3	ART_AND_DESIGN	12.281389	50000000+	Free	0.0	Teen	
4	ART_AND_DESIGN	6.875232	100000+	Free	0.0	Everyone	
...	...	...	...	...	...	...	
10833	BOOKS_AND_REFERENCE	3.806662	1000+	Free	0.0	Everyone	
10834	FAMILY	2.079442	500+	Free	0.0	Everyone	
10836	FAMILY	3.663562	5000+	Free	0.0	Everyone	
10837	FAMILY	1.609438	100+	Free	0.0	Everyone	
10840	LIFESTYLE	12.894981	10000000+	Free	0.0	Everyone	

7424 rows × 6 columns

Next steps:

[Generate code with independent_features](#)[New interactive sheet](#)

```
x=independent_features
y=target
```

```
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=1)
```

```
from sklearn.preprocessing import StandardScaler
standardization=StandardScaler()
df_std=standardization.fit_transform(df[num_cols])
df_std=pd.DataFrame(df_std)
df_std.columns=num_cols
df_std
```

	Reviews	Price	
0	-0.694347	-0.063058	
1	-0.206386	-0.063058	
2	1.014637	-0.063058	
3	1.259116	-0.063058	
4	-0.206386	-0.063058	
...	...	...	
7419	-1.038215	-0.063058	
7420	-1.506431	-0.063058	
7421	-1.077007	-0.063058	
7422	-1.633839	-0.063058	
7423	1.425449	-0.063058	

7424 rows × 2 columns

Next steps:

[Generate code with df_std](#)[New interactive sheet](#)

```
df[['Reviews', 'Price']] = df_std[['Reviews', 'Price']]
```

df

	Category	Reviews	Installs	Type	Price	Content Rating	Rating_category
0	ART_AND_DESIGN	-0.694347	10000+	Free	-0.063058	Everyone	high
1	ART_AND_DESIGN	-0.206386	500000+	Free	-0.063058	Everyone	high
2	ART_AND_DESIGN	1.014637	5000000+	Free	-0.063058	Everyone	high
3	ART_AND_DESIGN	1.259116	50000000+	Free	-0.063058	Teen	high
4	ART_AND_DESIGN	-0.206386	100000+	Free	-0.063058	Everyone	high
...	...	...	...	...	...	...	...
10833	BOOKS_AND_REFERENCE	NaN	1000+	Free	NaN	Everyone	high
10834	FAMILY	NaN	500+	Free	NaN	Everyone	high
10836	FAMILY	NaN	5000+	Free	NaN	Everyone	high
10837	FAMILY	NaN	100+	Free	NaN	Everyone	high
10840	LIFESTYLE	NaN	10000000+	Free	NaN	Everyone	high

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df1=pd.read_csv('/content/honeyproduction (1).csv')
df1
```

	state	numcol	yieldpercol	totalprod	stocks	priceperlb	prodvalue	year	
0	AL	16000.0	71	1136000.0	159000.0	0.72	818000.0	1998	
1	AZ	55000.0	60	3300000.0	1485000.0	0.64	2112000.0	1998	
2	AR	53000.0	65	3445000.0	1688000.0	0.59	2033000.0	1998	
3	CA	450000.0	83	37350000.0	12326000.0	0.62	23157000.0	1998	
4	CO	27000.0	72	1944000.0	1594000.0	0.70	1361000.0	1998	
...	...	...	...	...	...	...	...	...	
621	VA	4000.0	41	164000.0	23000.0	3.77	618000.0	2012	
622	WA	62000.0	41	2542000.0	1017000.0	2.38	6050000.0	2012	
623	WV	6000.0	48	288000.0	95000.0	2.91	838000.0	2012	
624	WI	60000.0	69	4140000.0	1863000.0	2.05	8487000.0	2012	
625	WY	50000.0	51	2550000.0	459000.0	1.87	4769000.0	2012	

626 rows × 8 columns

Next steps:

[Generate code with df1](#)[New interactive sheet](#)

df1.sample(5)

	state	numcol	yieldpercol	totalprod	stocks	priceperlb	prodvalue	year	
309	HI	9000.0	131	1179000.0	283000.0	1.43	1686000.0	2005	
65	NE	58000.0	77	4466000.0	1965000.0	0.61	2724000.0	1999	
69	NY	69000.0	70	4830000.0	2077000.0	0.66	3188000.0	1999	
454	OR	50000.0	43	2150000.0	1097000.0	1.41	3032000.0	2008	
335	TX	84000.0	71	5964000.0	954000.0	0.86	5129000.0	2005	

df1.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 626 entries, 0 to 625
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   state           626 non-null    object
1   numcol          626 non-null    float64
2   yieldpercol     626 non-null    int64
3   totalprod       626 non-null    float64
4   stocks          626 non-null    float64
5   priceperlb      626 non-null    float64
6   prodvalue       626 non-null    float64
7   year            626 non-null    int64
dtypes: float64(5), int64(2), object(1)
memory usage: 39.3+ KB
```

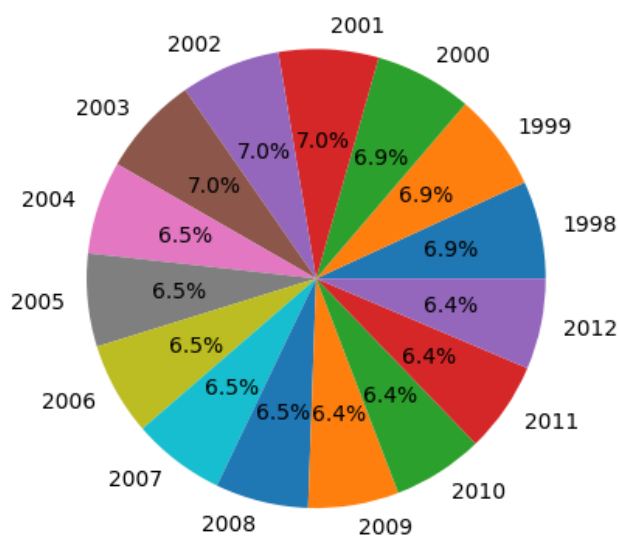
df1.shape

(626, 8)

df1.describe()

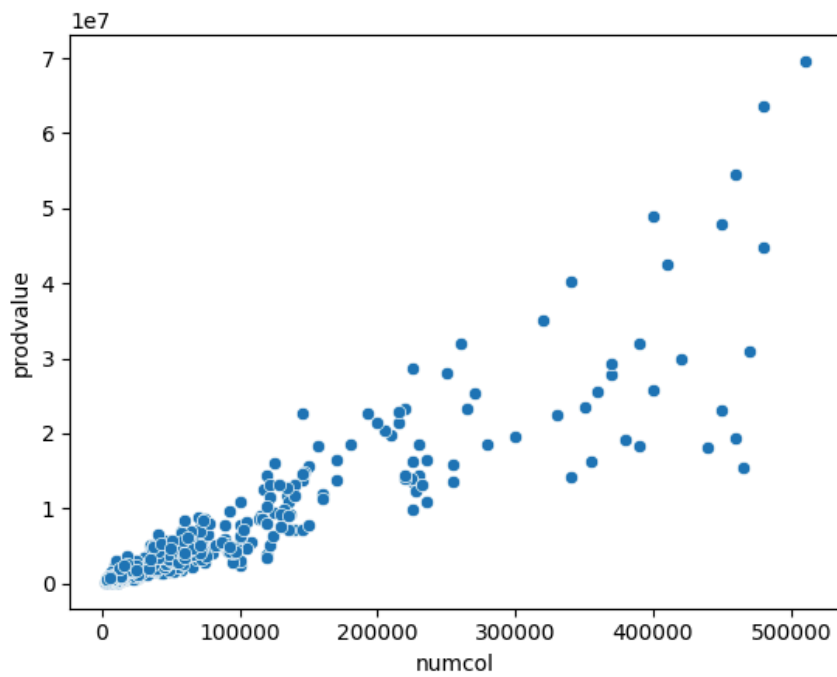
	numcol	yieldpercol	totalprod	stocks	priceperlb	prodvalue	year
count	626.000000	626.000000	6.260000e+02	6.260000e+02	626.000000	6.260000e+02	626.000000
mean	60284.345048	62.009585	4.169086e+06	1.318859e+06	1.409569	4.715741e+06	2004.864217
std	91077.087231	19.458754	6.883847e+06	2.272964e+06	0.638599	7.976110e+06	4.317306
min	2000.000000	19.000000	8.400000e+04	8.000000e+03	0.490000	1.620000e+05	1998.000000
25%	9000.000000	48.000000	4.750000e+05	1.430000e+05	0.932500	7.592500e+05	2001.000000
50%	26000.000000	60.000000	1.533000e+06	4.395000e+05	1.360000	1.841500e+06	2005.000000
75%	63750.000000	74.000000	4.175250e+06	1.489500e+06	1.680000	4.703250e+06	2009.000000
max	510000.000000	136.000000	4.641000e+07	1.380000e+07	4.150000	6.961500e+07	2012.000000

```
plt.pie(df1.groupby('year').size(), autopct='%1.1f%%', labels=df1.groupby('year').size().index)
plt.show()
```



```
sns.displot(df1['priceperlb'], kind='hist')
plt.show()
```

```
sns.scatterplot(df1,x='numcol',y='prodvalue')  
plt.show()
```



```
sns.boxplot(df1,x='year',y='prodvalue')
```

```
<Axes: xlabel='year' ylabel='prodvalue'>
```